Reinforcement Learning-Based Computer Numerical Control Motion Planning with Adaptive Preview and Jerk-Constrained Action Projection

Abstract

Computer numerical control (CNC) motion planning converts CAM-generated tool paths into executable interpolation trajectories that satisfy contour tolerance and kinematic limits. Conventional smooth-then-schedule pipelines determine the geometric path before regulating the feedrate. After the path has been fixed, the feedrate planner can no longer use the tolerance band to adjust motion at corners and curvature-transition regions. This separation reduces the coupling between geometric smoothing and dynamic feasibility. We propose J-NNC, a closed-loop reinforcement learning framework that treats tolerance-band trajectory shaping and feedrate regulation as one constrained decision problem. Rather than selecting a path and then assigning its timing, J-NNC learns local motion decisions within the tolerance band and projects them onto the kinematically feasible set, thereby linking geometric freedom with jerk-limited execution. Experiments on square, circular, and butterfly-shaped paths show that J-NNC completes all cases with full path progress and no linear-jerk limit exceedance. It removes the violations observed in the neural network controller (NNC) baseline and the no-KCM ablation. Compared with a traditional two-step baseline, J-NNC achieves a shorter execution time, a higher active jerk reach rate, and more effective tolerance-band utilization while preserving jerk feasibility.

Keywords: computer numerical control, reinforcement learning, Adaptive Preview, jerk-constrained action projection

1. Introduction

Conventional CNC motion planning usually follows a serial pipeline: the tool path is smoothed first, and feedrate planning is then carried out along the smoothed path [1, 2]. This procedure improves the geometric continuity of CAM-generated tool paths. Once the smoothed trajectory is fixed, however, the feedrate planner can mainly adjust the motion speed along that trajectory [3, 4]. It has little room to reuse the spatial freedom inside the contour-tolerance band, especially in corner regions and curvature-transition regions [5, 6]. For this reason, geometric smoothness, feedrate efficiency, and kinematic feasibility under velocity, acceleration, and jerk limits are difficult to optimize together in high-precision, high-speed machining [1, 7].

In practice, this serial pipeline is built mainly from local smoothing, global smoothing, and feedrate scheduling. Local smoothing constructs transition curves between adjacent blocks. It is well suited to real-time interpolation because the computation is local and the contour error can be bounded explicitly [8]. Higher-order Bézier, B-spline, and curvature-smooth transitions can further improve acceleration or jerk continuity around corners [9, 10]. Dense short segments and consecutive corners, however, often leave only

a limited transition length [11]. Global smoothing reconstructs several segments over a larger domain and improves continuity for dense short-block paths [12]. Under strict contour-error constraints, however, computational cost, iterative fitting, local feature preservation, and real-time implementation remain difficult [13]. Feedrate scheduling imposes velocity, acceleration, and jerk bounds along a selected path, but it usually starts from a predetermined geometric trajectory [7]. NURBS and S-shaped schemes improve feasibility under chord-error and axial-drive constraints [14]. Look-ahead interpolation uses upcoming path information to regulate feedrate over short-line or spline tool paths [15, 16]. These studies provide the main technical basis of the conventional serial CNC motion-planning pipeline.

Although this line of work has made substantial progress, the serial decomposition still separates geometry selection from timing regulation, even though these two decisions are physically coupled during interpolation. Local and global smoothing first determine the geometric curve used inside the contour-tolerance band, including the corner-transition shape and the curvature distribution. Feedrate scheduling then assigns a speed profile to the selected curve. The later stage can reduce or increase the feedrate, but it has limited ability to change how the tolerance band is used geometrically. This separation is especially restrictive when short segments, consecutive corners, and rapidly varying curvature appear in the same CAM path [11, 17]. A conservative smoothing result may cause unnecessary deceleration, while an aggressive transition may still require speed reduction to satisfy acceleration or jerk limits [5, 6]. Thus, a fixed serial procedure leaves limited room to balance the geometric use of the tolerance band with jerk-limited executable motion [1, 7].

To address the limitations of the conventional serial pipeline, one-step methods aim to perform trajectory smoothing and feedrate planning within a unified planning process. Direct axis-velocity blending and integrated acceleration/deceleration interpolation plan axis motion near corners under real-time constraints [6, 18]. Minimum-time cornering and high-speed cornering with vibration suppression optimize corner traversal under contour-error and actuator limits [5, 19]. Driving-force-constrained smoothing brings drive capability into corner planning, and quadratic B-spline fitting combines tool-path construction with optimal velocity interpolation under bounded error [13, 20]. Integrated smoothing methods for line–arc and five-axis paths further handle transition overlap and feedrate fluctuation within unified smoothing procedures [21, 22]. In a closely related direction, Li et al. proposed neural-network numerical control (NNC), which treats CNC trajectory smoothing as a closed-loop action-decision problem and directly generates angle-change and step-length-change commands at each interpolation cycle [11]. NNC shows that trajectory generation inside the tolerance band can be learned as an online decision process, with higher machining efficiency than local smoothing and higher contour accuracy than global smoothing [11].

These one-step methods strengthen the connection between path geometry and speed behavior, but several issues remain for tolerance-band CNC motion planning. Many optimization-based methods rely on predefined transition forms, local corner models, nonlinear constraints, or iterative computation. Their performance can therefore be sensitive to segment length, corner distribution, and local geometry variation [13, 21]. In NNC, the neural-network action is converted directly into an interpolation position or a servo command. If velocity, acceleration, and jerk constraints are not handled explicitly, or if no projection is applied at the execution layer, the generated command may violate the kinematic limits of the machine tool [11]. In addition, the reward design mainly

focuses on contour error, while path progress, direction consistency, motion smoothness, and stage-dependent behavior near corners receive less attention [11]. These limitations make it difficult for one-step planning to use the tolerance band effectively, maintain path advancement, and guarantee executable interpolation under jerk constraints at the same time.

This paper addresses these issues by proposing J-NNC (Jerk-constrained Neural Numerical Controller), a reinforcement learning-based CNC motion-planning framework with jerk-constrained action projection and Adaptive Preview. The framework formulates mixed-tool-path motion planning as a constrained one-step decision problem within the contour-tolerance band. A kinematic constraint module (KCM) is inserted between the learned policy and the execution layer. It maps unconstrained steering and feedrate intentions to interpolation control variables that satisfy velocity, acceleration, and jerk constraints. The policy is trained using PPO [23] and outputs steering, feedrate, and Adaptive Preview adjustment actions in a closed loop. The state representation includes tracking, kinematic, corner, and multi-scale forward-geometric information, and the reward accounts for path progress, contour error, direction consistency, and motion smoothness. Fig. 1 compares conventional serial motion planning with the closed-loop J-NNC motion-planning framework.

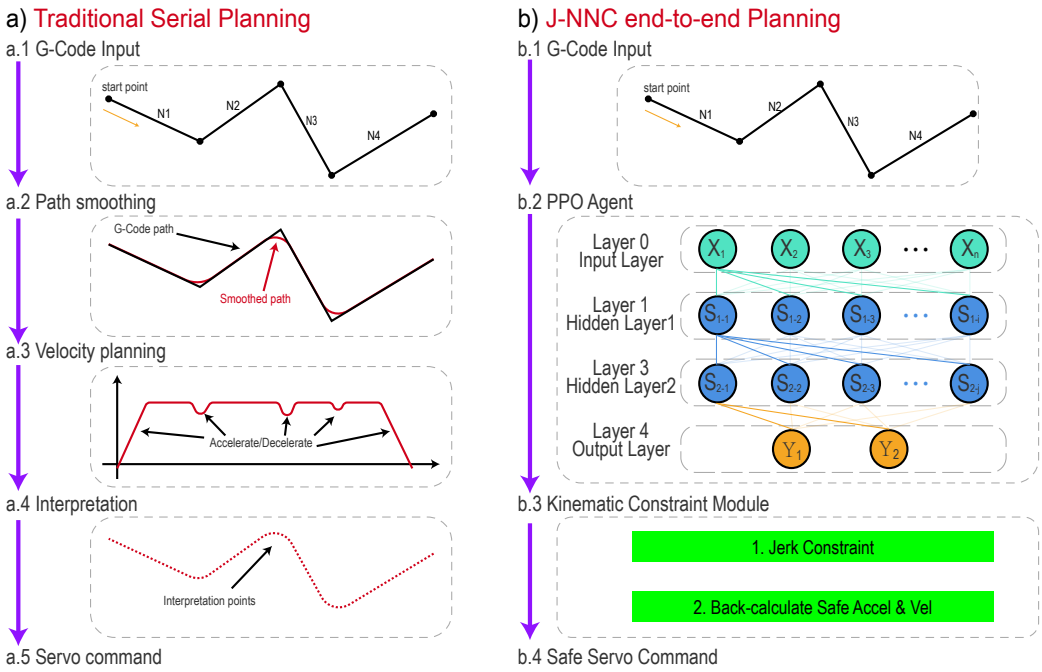

Fig. 1: Comparison of conventional serial motion planning and the J-NNC motion-planning framework. J-NNC denotes the Jerk-constrained Neural Numerical Controller, a neural-network-based CNC motion planner with jerk-constrained action projection.

The main contributions are as follows:

(1) A KCM is developed to project raw steering and feedrate outputs into executable commands while satisfying velocity, acceleration, and jerk limits for both linear and angular motion.

(2) A tolerance-band one-step decision formulation with Adaptive Preview is proposed

to jointly learn trajectory shaping and feedrate regulation from structured geometric and kinematic states.

(3) Experiments on square, circular, and butterfly-shaped paths show that J-NNC completes the full path and removes the linear-jerk-limit violations observed in the NNC baseline and the no-KCM ablation.

The remainder of this paper is organized as follows. Section 2 defines the problem and models the constraints. Section 3 presents the proposed method. Section 4 reports the experimental settings, comparative results, and ablation analysis. Section 5 concludes the paper.

2. Problem Formulation

This section defines the motion-planning problem considered in this study, together with the geometric and execution constraints used in the proposed method. These definitions provide the basis for the method described in the following sections.

2.1. Tool Path and Geometric Feasible Region

Consider a discrete sequence of cutter-location points generated by a CAM system,

$$P_{\text{raw}} = p_1, p_2, \ldots, p_N, \tag{1}$$

where $p_i \in \mathbb{R}^2$ denotes a reference point in the interpolation plane. Let ε denote the prescribed contour tolerance. The reference path is offset by $\varepsilon/2$ on both sides to define the geometric feasible region in which the policy may locally deviate from the reference path:

$$\Omega = x \mid d(x, P_{\text{raw}}) \leq \varepsilon/2, \tag{2}$$

where $d(x, P_{\text{raw}})$ denotes the minimum distance from point x to the reference path. This region allows local redistribution of the trajectory within the tolerance band, which helps smooth corner traversal while preserving contour accuracy.

For closed-loop decision making, the reference path is parameterized by arc length. At the current projected arc length ℓ_t, local geometric quantities are extracted, including the normal error, tangential direction, distance to the next corner, and turning angle of the next corner. These quantities are used in the state representation, reward weighting, and corner-mode identification.

2.2. Constrained Action Decision Formulation

At each interpolation period Δt, the policy receives the current state s_t and outputs a normalized action a_t^{π}. The kinematic constraint module (KCM) projects this action into an executable control command, which is then applied to the environment. The environment updates the position, orientation, and kinematic state, and returns the immediate reward r_t. The task is therefore written as a constrained sequential decision-making problem:

$$\max_{\pi} \ \mathbb{E}\pi \left[\sum t = 0^T \gamma^t r_t \right], \tag{3}$$

subject to the geometric constraint

$$x_t \in \Omega, \tag{4}$$

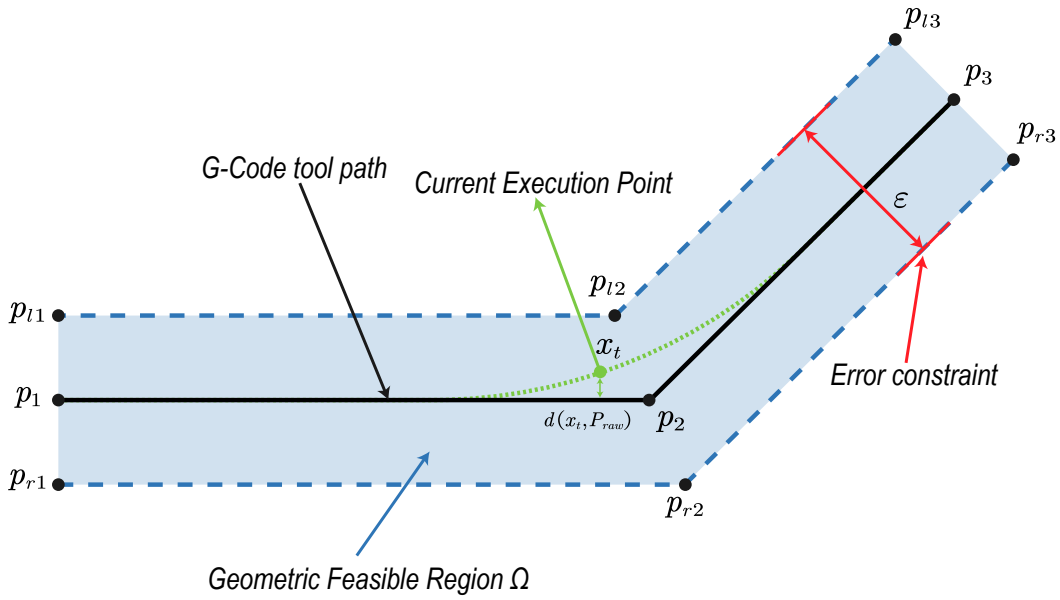

Fig. 2: Geometric feasible region formed by the reference path and the contour tolerance band.

and the execution-side kinematic constraints

$$|v_t| \le V_{\max}, \quad |a_t| \le A_{\max}, \quad |j_t| \le J_{\max}, \tag{5}$$

$$|\omega_t| \le \Omega_{\max}, \quad |\dot{\omega} t| \le A\max^{\omega}, \quad |\ddot{\omega} t| \le J\max^{\omega}. \tag{6}$$

Here, v_t, a_t, and j_t denote the linear velocity, linear acceleration, and linear jerk, respectively, whereas ω_t, $\dot{\omega}_t$, and $\ddot{\omega}_t$ denote the angular velocity, angular acceleration, and angular jerk, respectively.

3. Methodology

3.1. Overall Framework

In this paper, Adaptive Preview denotes the state-dependent preview mechanism of J-NNC, whereas look-ahead is retained only when referring to conventional CNC look-ahead interpolation or prior work.

Motion planning for hybrid tool paths is formulated as a closed-loop decision process in a reinforcement learning setting. The policy network acts as the agent, and the tool-path simulation system, which accounts for contour tolerance and kinematic constraints, acts as the environment. At each interpolation cycle, the environment builds the state s_t from the current execution position, kinematic state, local corner information, and Adaptive Preview geometry. Given s_t, the policy π_θ produces an action a_t that gives the steering, feedrate, and Adaptive Preview adjustment commands for the current cycle. The action is corrected by the KCM and then applied to the environment. The environment moves to the next state s_{t+1} and returns a reward r_t based on path progress, contour error, and motion smoothness. During training, reward feedback updates the policy parameters, enabling the policy to learn a control law for trajectory smoothing, feedrate planning, and kinematic-constraint satisfaction within the tolerance band.

Fig. 3 shows the overall framework of the proposed method. Given a reference path and a contour tolerance, the environment carries out path projection, corner scanning, and Adaptive Preview geometry extraction to form a structured state vector at the current

time step. The policy network outputs a normalized action. The steering and feedrate components are mapped to the pre-projection motion control variables $(\omega_t^{pre}, v_t^{pre})$, and the Adaptive Preview component is mapped to the current Adaptive Preview distance L_t. The KCM then uses the velocity, acceleration, and jerk states from the previous time step to project $(\omega_t^{pre}, v_t^{pre})$ onto the feasible kinematic set. This projection gives the executable control variables (ω_t, v_t), which satisfy the velocity, acceleration, and jerk limits. The execution result updates the current position, kinematic quantities, corner phase, and Adaptive Preview observations. During training, the same result also enters the policy update through the reward signal. The whole procedure forms a closed reinforcement learning loop for motion planning.

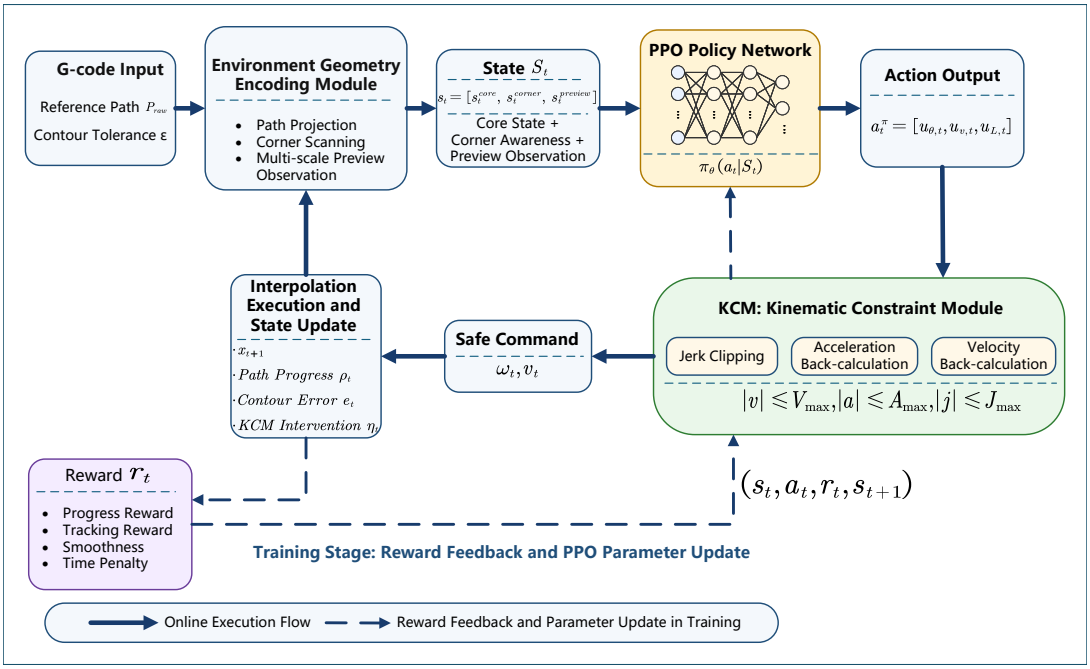

Fig. 3: Overall closed-loop framework of the proposed method.

3.2. State Design

A hierarchical structured state representation is used:

$$s_t = [s_t^{core},\ s_t^{corner},\ s_t^{\mathrm{AP}}]. \tag{7}$$

Here, s_t^{core} describes the local tracking and kinematic state at the current time step, s_t^{corner} describes the relation between the current path segment and the next corner, and s_t^{AP} records the geometric trend over several Adaptive Preview distances ahead.

The core state is an eight-dimensional vector:

$$s_t^{core} = [\bar{e}_t,\ \bar{e}_{n,t},\ \bar{\tau}_t,\ \bar{v}_t,\ \bar{a}_t,\ \bar{\omega}_t,\ \rho_t,\ \bar{d}_{c,t}], \tag{8}$$

where $\bar{e}_t$ is the normalized contour error, $\bar{e}_{n,t}$ is the signed normal error, and $\bar{\tau}_t$ is the normalized angular error between the current heading and the local tangent direction. The terms $\bar{v}_t$, $\bar{a}_t$, and $\bar{\omega}_t$ are the normalized linear velocity, linear acceleration, and angular velocity, respectively. In addition, $\rho_t \in [0, 1]$ gives the overall path progress, and $\bar{d}_{c,t}$ is the normalized distance to the next corner.

The corner-aware state is a four-dimensional feature vector:

$$s_t^{corner} = [\bar{\phi}_t,\ \sigma_t,\ m_t,\ \sigma_t \bar{e}_{n,t}], \tag{9}$$

where $\bar{\phi}_t$ is the normalized turning angle of the next corner, $\sigma_t \in -1, 0, 1$ is the turning sign of the corner, and $m_t \in 0, 1$ indicates whether the current position is within the corner-influence interval. The last term, $\sigma_t \bar{e}_{n,t}$, encodes the signed positional relation to the inner or outer side of the turn.

For Adaptive Preview observation, several Adaptive Preview scales $\lambda_k k = 1^{N_L}$ are defined along the arc length of the reference path. Let ℓ_t be the arc-length coordinate of the projection of the current execution point onto the reference path. The forward observation point for the k-th Adaptive Preview scale is

$$q_t^{(k)} = P\mathrm{raw}(\ell_t + \lambda_k L_t), \qquad k = 1, \ldots, N_L, \tag{10}$$

where λ_k is the scale coefficient and L_t is the base Adaptive Preview distance used in the current cycle. In this formulation, the near-, medium-, and far-range Adaptive Preview points are determined along the arc length of the reference path, rather than from ray intersections along the current execution direction. For each scale, the angular variation of the forward observation point relative to the current tangent direction and the corresponding distance ratio are extracted:

$$s_t^{\mathrm{AP}} = \left[\Delta\theta_t^{(1)},\ \bar{d}_t^{(1)},\ \ldots,\ \Delta\theta_t^{(N_L)},\ \bar{d}_t^{(N_L)}\right]. \tag{11}$$

Here, $\Delta\theta_t^{(k)}$ describes the directional change at the k-th Adaptive Preview scale, and $\bar{d}_t^{(k)}$ is the normalized distance from the current position to the corresponding Adaptive Preview point. The main implementation uses three Adaptive Preview scales.

3.3. Action Space and Kinematic Constraint Module

At each control cycle, the policy network outputs a normalized action:

$$a_t^{\pi} = [u_{\theta,t}, u_{v,t}, u_{L,t}], \tag{12}$$

where $u_{\theta,t} \in [-1, 1]$ is the normalized steering component, $u_{v,t} \in [0, 1]$ is the normalized feedrate component, and $u_{L,t} \in [0, 1]$ is the Adaptive Preview adjustment component. Given the admissible Adaptive Preview range $[L_{\min}, L_{\max}]$, the physical Adaptive Preview distance is mapped as

$$L_t = L_{\min} + u_{L,t}(L_{\max} - L_{\min}). \tag{13}$$

The policy output is first converted to pre-projection motion control variables in physical units:

$$\omega_t^{pre} = u_{\theta,t}\Omega_{\max}, \qquad v_t^{pre} = u_{v,t}V_{\max}. \tag{14}$$

The Adaptive Preview distance L_t is used to compute the local reference direction and Adaptive Preview geometry. The KCM then projects $(\omega_t^{pre}, v_t^{pre})$ onto the kinematically feasible set according to the kinematic state at the previous time step. For the linear-velocity channel, let v_{t-1} and a_{t-1} be the linear velocity and linear acceleration in the previous cycle. The pre-projection acceleration is

$$a_t^{pre} = \frac{v_t^{pre} - v_{t-1}}{\Delta t}, \tag{15}$$

and the corresponding pre-projection jerk is

$$j_t^{pre} = \frac{a_t^{pre} - a_{t-1}}{\Delta t}. \tag{16}$$

The KCM clips this jerk by the jerk bound:

$$j_t = \mathrm{clip}(j_t^{pre}, -J_{\max}, J_{\max}), \tag{17}$$

and then reconstructs the realizable acceleration and velocity for the current cycle:

$$a_t = \mathrm{clip}(a_{t-1} + j_t \Delta t, -A_{\max}, A_{\max}), \tag{18}$$

$$v_t = \mathrm{clip}\left(v_{t-1} + a_{t-1}\Delta t + \frac{1}{2} j_t \Delta t^2, 0, V_{\max}\right). \tag{19}$$

The angular-velocity channel uses the same projection procedure. The resulting (ω_t, v_t) is then applied to the environment as the executable control input. The resulting online projection procedure is summarized in Algorithm 1.

Algorithm 1 KCM-constrained action projection

Require: Policy output $a_t^{\pi} = [u_{\theta,t}, u_{v,t}, u_{L,t}]$; previous-step kinematic state $(v_{t-1}, a_{t-1}, \omega_{t-1}, \alpha_{t-1})$

Ensure: Executed control variables (ω_t, v_t) and the updated kinematic state

1: Map $u_{\theta,t}$ and $u_{v,t}$ to the unconstrained motion commands $(\omega_t^{pre}, v_t^{pre})$
2: Map $u_{L,t}$ to the Adaptive Preview distance L_t
3: Compute the unconstrained linear acceleration a_t^{pre} from v_t^{pre} and v_{t-1}
4: Compute the unconstrained linear jerk j_t^{pre} from a_t^{pre} and a_{t-1}
5: Clip j_t^{pre} to obtain j_t under the prescribed jerk bound
6: Recover the linear acceleration a_t from j_t and apply acceleration clipping
7: Integrate (v_{t-1}, a_{t-1}, j_t) to obtain the linear velocity v_t for the current control cycle
8: Apply the same procedure to the angular-velocity channel to obtain the constrained angular velocity ω_t
9: Return the executed control variables (ω_t, v_t) and the updated kinematic state

The intervention degree of the KCM is defined as

$$\eta_t = \frac{|v_t - v_t^{pre}|}{V_{\max}} + \frac{|\omega_t - \omega_t^{pre}|}{\Omega_{\max}}. \tag{20}$$

This scalar is not part of the primary optimization objective. It is used only as an auxiliary metric in the result analysis.

3.4. Reward Design

The instantaneous reward is

$$r_t = r_t^{prog} + r_t^{track} + r_t^{dir} + r_t^{smooth}. \tag{21}$$

The progress reward is defined as

$$r_t^{prog} = w_p \max(0, \rho_t - \rho_{t-1}), \tag{22}$$

where ρ_t is the overall path progress.

The contour-error penalty is defined as

$$\phi_e(e_t) = \begin{cases} 0, & |e_{n,t}| \le \delta_t, \\ \left(\dfrac{|e_{n,t}| - \delta_t}{\varepsilon/2} \right)^2, & |e_{n,t}| > \delta_t, \end{cases} \tag{23}$$

and the corresponding tracking reward is

$$r_t^{track} = -w_e(c_t)\phi_e(e_t). \tag{24}$$

Here, δ_t is the relaxed boundary within the tolerance band, and $c_t \in [0, 1]$ is the local corner intensity. They are defined as

$$\delta_t = \kappa_\delta \frac{\varepsilon}{2} \mathbf{1}(m_t = 1), \tag{25}$$

$$c_t = \text{clip}\left((1 - \alpha_c)c_{t-1} + \alpha_c \frac{|\Delta\theta_t^{(N_L)}|}{\theta_0}, 0, 1 \right), \tag{26}$$

where κ_δ is the error-relaxation ratio, m_t indicates whether the current state is within the corner-influence region, $\Delta\theta_t^{(N_L)}$ is the Adaptive Preview directional change at the farthest scale, θ_0 is the normalization angle for local corner intensity, and $\alpha_c = 1/T_c$ is the exponential moving-average coefficient. In the main experiments, κ_δ is set to 0. Thus, the contour-error term has no additional dead zone, while c_t is still used to adjust the tracking and smoothness weights.

The directional consistency term is defined as

$$r_t^{dir} = -w_\tau |\tau_t|, \tag{27}$$

where τ_t is the angular error between the current heading and the local reference direction.

The smoothness term is defined as

$$r_t^{smooth} = -w_s(c_t)\psi_t, \tag{28}$$

where ψ_t can be a normalized measure of linear jerk, angular acceleration, or angular jerk. The weight $w_s(c_t)$ is adjusted according to the local corner intensity. One implementation is

$$w_s(c_t) = w_s^0 c_t^\gamma, \qquad \gamma > 1, \tag{29}$$

where $w_s(c_t)$ increases with c_t.

3.5. PPO Training Configuration

The policy is trained with Proximal Policy Optimization (PPO). The Actor is a shared multilayer perceptron with three hidden layers of widths 256, 128, and 64. Each layer uses ELU activation followed by LayerNorm. Separate output heads parameterize the mean and standard deviation of a three-dimensional action distribution. The steering mean is mapped to $[-1, 1]$ with tanh, while the feedrate and Adaptive Preview mean are mapped to $[0, 1]$ with the sigmoid function. The standard deviation is obtained by applying the softplus function and then adding 10^{-3} for numerical stability. The Critic has layer widths $256, 128, 64, 1$ and uses ELU activation, LayerNorm, and a dropout rate of 0.1. The main training hyperparameters are listed in Tab. 1.

Tab. 1: PPO hyperparameters and training protocol.

Parameter	**Value**
Actor learning rate	3.0×10^{-4}
Critic learning rate	1.5×10^{-4}
Discount factor γ	0.99
GAE parameter λ	0.95
PPO clip coefficient	0.1
Number of epochs per update	8
Entropy coefficient	0.01
Gradient clipping	0.5

4. Experiments and Results

We evaluate J-NNC in simulation on three paths with different geometric demands. The tests focus on three practical requirements of CNC interpolation: the tool should reach the end of the commanded path, local transitions should remain within the contour-tolerance band, and the executed commands should satisfy the prescribed velocity, acceleration, and jerk limits. The main comparison uses the neural network controller (NNC) baseline and a traditional two-step baseline. The ablation study then examines Adaptive Preview, KCM, Adaptive Preview observation, and reward scheduling.

4.1. Experimental Settings

The three test paths are selected to stress different parts of the planner. The square path emphasizes sharp-corner deceleration, turning, and re-acceleration. The circular path tests continuous turning under sustained contour control. The butterfly-shaped path contains mixed curvature changes and local geometric variation.

Unless otherwise stated, the J-NNC and ablation evaluations use a contour-tolerance band of $\varepsilon = 2.5$ mm, an interpolation period of $\Delta t = 0.001$ s, and a maximum episode length of 60000 steps, corresponding to 60 s. The linear kinematic limits are $V_{\max} = 100$ mm/s, $A_{\max} = 2000$ mm/s^2, and $J_{\max} = 62500$ mm/s^3. The Adaptive Preview distance is selected within $[0.8, 4.6]$ mm, and the multi-scale observation uses scales $(0.5, 1.0, 2.0)$ with 8 forward points.

The main comparison includes J-NNC and two baselines. The NNC baseline directly executes the learned policy output, following Li et al. [11]. The traditional two-step baseline smooths the path first and then applies jerk-limited feedrate scheduling. We report path advancement, contour error, execution time, and satisfaction of the kinematic constraints.

Each ablation modifies one part of the proposed method. The fixed Adaptive Preview setting replaces the state-dependent Adaptive Preview action with a fixed Adaptive Preview distance. The no-KCM setting removes the execution-layer kinematic-constraint projection and directly applies the policy output in the environment. The setting without Adaptive Preview observation removes the multi-scale forward geometric input. The no-dual-reward setting disables the separate reward scheduling for straight and corner segments. These variants separate the effects of Adaptive Preview, execution-side constraints, forward geometric information, and reward structure.

Because the task is constrained CNC motion planning, a shorter execution time is useful only when both contour and jerk constraints are satisfied.

4.2. Main Comparative Results

The main comparison is designed to answer two questions. The comparison with NNC asks whether learned direct control is sufficient without an execution-layer projection. The comparison with the traditional two-step baseline asks whether closed-loop planning inside the tolerance band can use the dynamic limits more effectively than a serial smooth-then-schedule pipeline. The two-step baseline first generates a fixed corner-smoothed path and then applies conservative jerk-limited feedrate scheduling along that path, following the strategy used in local corner smoothing methods [24].

Tab. 2: Comparison among J-NNC, the NNC baseline, and the traditional two-step baseline on representative evaluation trajectories.

Path	Metric	J-NNC	NNC baseline	Traditional two-step
square	Termination status	success	success	success
	Final progress	1.000	1.000	1.000
	Maximum contour error [mm]	0.834	0.676	0.309
	Maximum relative linear-jerk exceedance	0.000	3199.000	0.000
	Termination time [s]	12.192	8.672	24.780
circle	Termination status	success	max_steps	success
	Final progress	1.000	0.990	1.000
	Maximum contour error [mm]	0.096	6.259	0.000
	Maximum relative linear-jerk exceedance	0.000	3199.000	0.000
	Termination time [s]	9.117	60.000	17.499
butterfly	Termination status	success	success	success
	Final progress	1.000	1.000	1.000
	Maximum contour error [mm]	0.745	0.207	0.012
	Maximum relative linear-jerk exceedance	0.000	3199.000	0.000
	Termination time [s]	10.480	5.807	15.360

Tab. 3: Efficiency and dynamic-constraint utilization metrics of J-NNC and the traditional two-step baseline.

Path	Method	Mean feedrate utilization	P95 linear-jerk utilization	Active jerk reach rate
square	J-NNC	0.328	1.000	0.991
	Traditional two-step	0.161	0.000	0.000
circle	J-NNC	0.345	1.000	0.929
	Traditional two-step	0.180	0.000	0.000
butterfly	J-NNC	0.221	1.000	0.635
	Traditional two-step	0.150	0.250	0.000

Tab. 2 reports path advancement, contour error, violation of the linear jerk limit, and termination time for the three methods. Tab. 3 separates feedrate utilization from dynamic-constraint utilization for J-NNC and the traditional two-step baseline. The active jerk reach rate should be read together with the maximum relative violation of the linear jerk limit, since high utilization is meaningful only when the prescribed limit is not exceeded.

The comparison with NNC mainly highlights dynamic feasibility. The NNC baseline reaches full progress on the square and butterfly paths, but its maximum relative violation of the linear jerk limit is 3199.000 on all three paths, and it fails to complete the circular path within the maximum step budget. The shorter NNC times on the square and butterfly paths therefore do not correspond to executable CNC motion under the

prescribed jerk bound. J-NNC completes all three paths with no violation of the linear jerk limit and also reduces the contour error on the circular path substantially relative to NNC.

J-NNC is also faster than the traditional two-step baseline while keeping the jerk constraint active. Both methods satisfy the jerk bound, but J-NNC reduces the termination time from 24.780 s to 12.192 s on the square path, from 17.499 s to 9.117 s on the circular path, and from 15.360 s to 10.480 s on the butterfly path. The utilization metrics in Tab. 3 explain the difference: J-NNC reaches higher mean feedrate utilization on all three paths and operates near the active jerk bound more often, whereas the two-step baseline remains conservative after geometric smoothing. This result supports the motivation for treating trajectory shaping and feedrate regulation as one closed-loop decision problem.

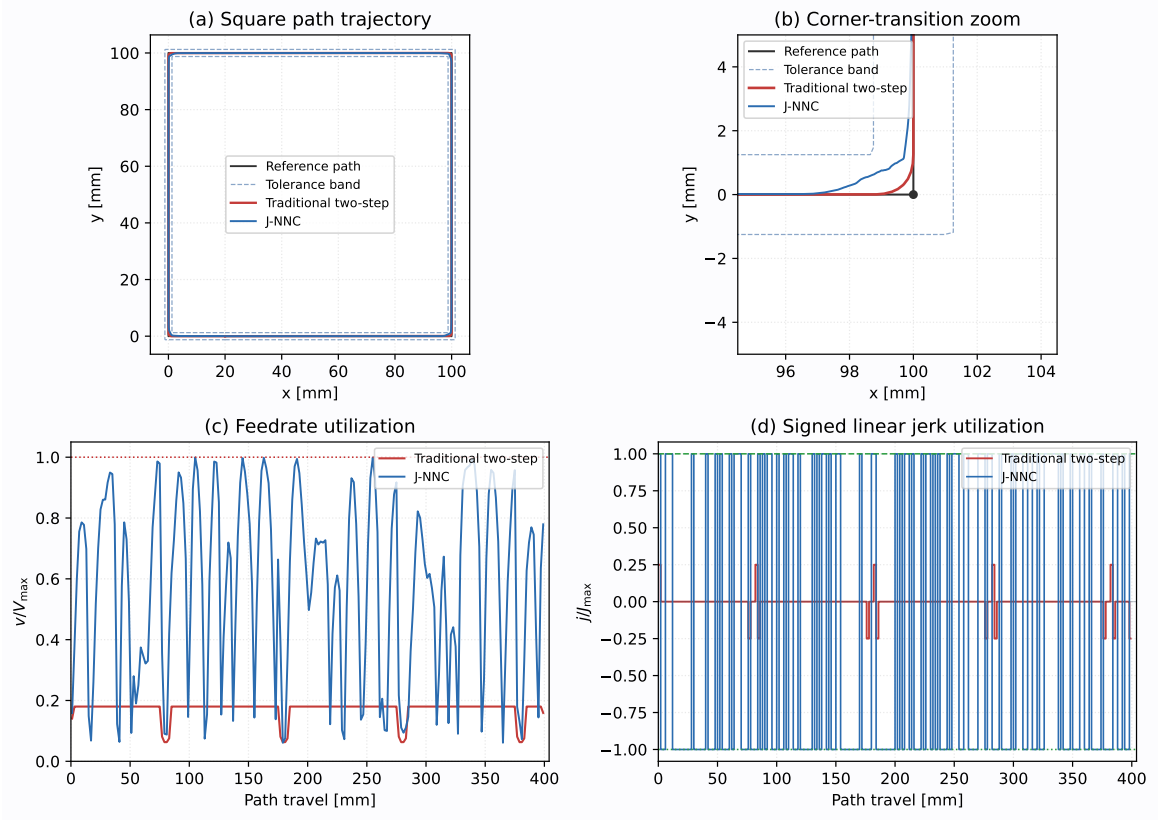

Fig. 4: Square-path comparison between J-NNC and the traditional two-step baseline: (a) trajectory within the tolerance band; (b) zoomed corner transition; (c) path-travel-aligned feedrate utilization $v/V_{\max}$; (d) path-travel-aligned signed linear-jerk utilization $j/J_{\max}$.

Fig. 4 gives a closer view of the square path. J-NNC begins the corner transition earlier within the tolerance band and maintains higher path-travel-aligned feedrate utilization. Its signed linear jerk remains within the prescribed $\pm J_{\max}$ bound throughout the run. The two-step baseline is also feasible, but its lower feedrate and jerk utilization lead to the longer termination time reported in Tab. 2.

Fig. 5 presents the trajectory comparisons on the three paths.

In Fig. 5(a)–(f), the J-NNC trajectories stay within the tolerance band but do not simply follow the reference polyline. On the square path, J-NNC forms local corner transitions, whereas NNC stays closer to the original polyline. On the circular path, J-NNC advances continuously with small contour error, whereas NNC does not complete the path. On the butterfly-shaped path, J-NNC introduces moderate offsets in regions

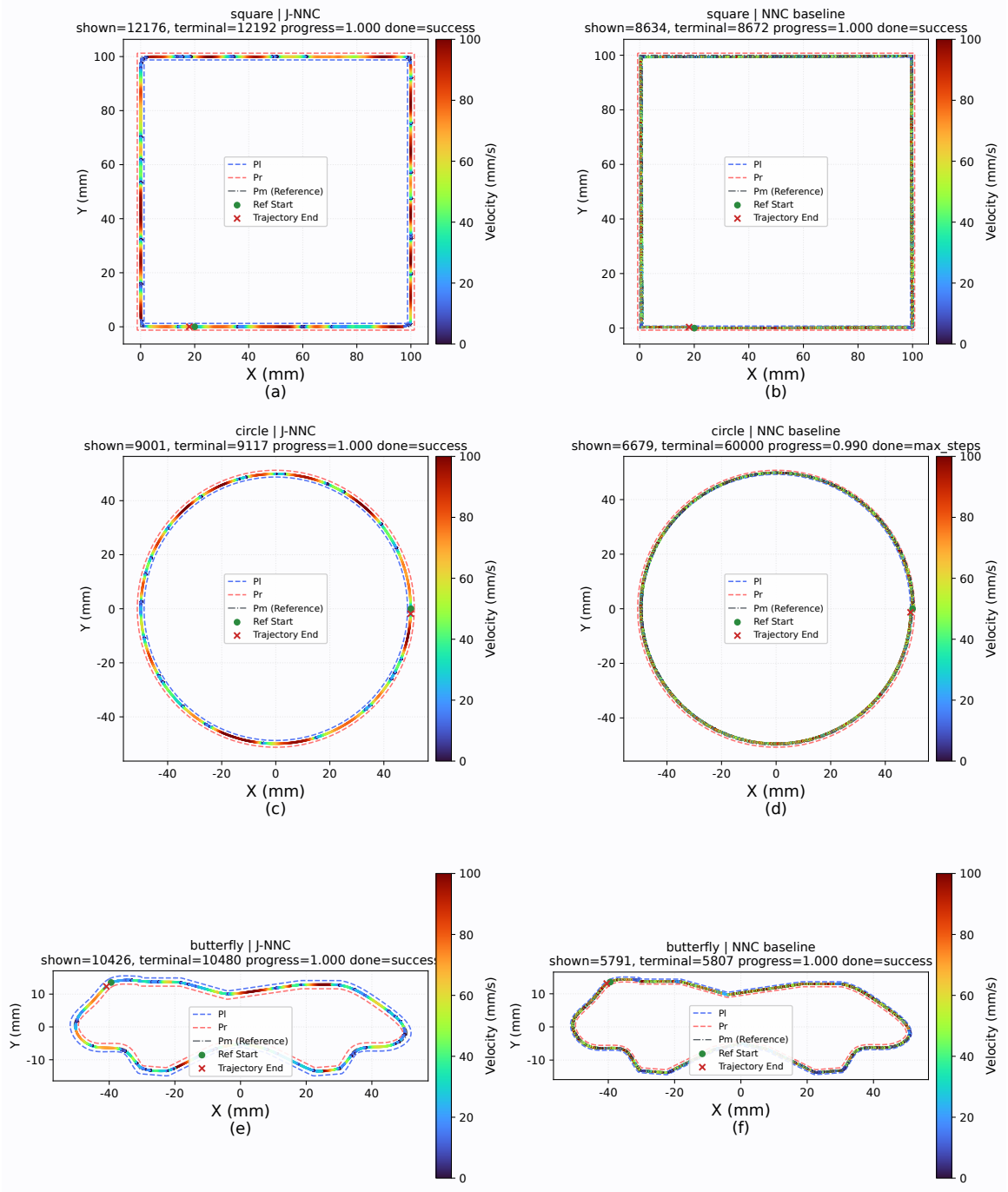

Fig. 5: Trajectory comparison between J-NNC and the NNC baseline on representative paths: (a) square path, J-NNC; (b) square path, NNC baseline; (c) circular path, J-NNC; (d) circular path, NNC baseline; (e) butterfly-shaped path, J-NNC; (f) butterfly-shaped path, NNC baseline.

where the curvature changes. These path-level results show how the tolerance band is used as controllable geometric freedom, and the square-corner region provides a clearer view of this behavior.

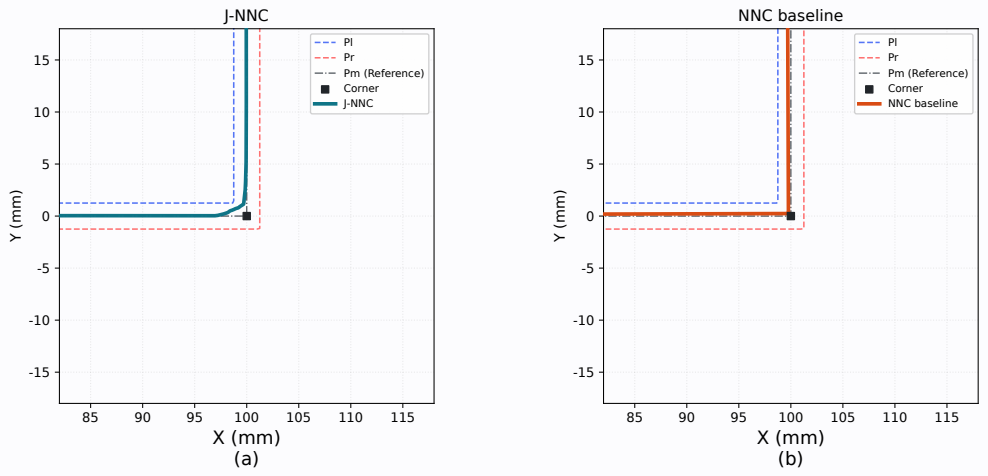

Fig. 6: Local zoomed-in comparison in the square-corner region: (a) J-NNC; (b) NNC baseline.

Fig. 6 shows that J-NNC begins the directional transition before reaching the vertex and distributes the turn inside the tolerance band. The NNC baseline remains closer to the two reference segments, so the change in direction is concentrated around the corner. This local behavior explains why the NNC trajectory can appear geometrically close to the polyline while still producing severe jerk-limit violations in Tab. 2. In contrast, J-NNC uses the admissible geometric margin to smooth the corner and then projects the learned command onto the feasible kinematic set, so the local transition remains executable.

Taken together, these comparisons separate the two advantages of J-NNC. Relative to NNC, it preserves learned path advancement while making the commands executable under the jerk bound; the corner zoom in Fig. 6 shows the local geometric mechanism behind this improvement. Relative to the traditional two-step baseline, it uses the tolerance band and the jerk limit more actively, shortening execution time without relaxing jerk feasibility.

4.3. Ablation Results

The ablation study changes Adaptive Preview, KCM, Adaptive Preview observation, and the straight-line/corner dual-reward scheme. Tab. 4 reports the termination status, final progress, maximum contour error, maximum relative violation of the linear jerk limit, and termination time for each path and configuration.

Tab. 4: Path-level reported evaluation results of the structured ablation study.

Configuration	Path	Status	Progress	Max contour error [mm]	Max jerk exceedance	Time [s]
J-NNC	square	success	1.000	0.834	0.000	12.192
	circle	success	1.000	0.096	0.000	9.117
	butterfly	success	1.000	0.745	0.000	10.480
Fixed Adaptive Preview	square	success	1.000	0.699	0.000	8.680
	circle	success	1.000	0.046	0.000	4.804
	butterfly	success	1.000	0.638	0.000	5.049
Without KCM	square	success	1.000	0.722	3199.000	10.301
	circle	success	1.000	0.061	3199.000	5.926
	butterfly	success	1.000	0.707	3199.000	7.446
Without Adaptive Preview observation	square	success	1.000	0.805	0.000	8.151
	circle	success	1.000	0.077	0.000	4.573
	butterfly	success	1.000	0.820	0.000	6.044
Without straight-line/corner dual rewards	square	success	1.000	0.824	0.000	15.132
	circle	success	1.000	0.030	0.000	10.388
	butterfly	success	1.000	0.918	0.000	9.623

Tab. 4 shows that all ablated configurations finish the reported evaluation runs, but they differ in termination time, contour error, and jerk feasibility. The no-KCM setting isolates the role of execution-layer projection. Without this projection, the controller still advances along the paths, but the jerk violation increases sharply. Because KCM is active in all other configurations, their maximum relative violation of the linear jerk limit remains zero.

Fig. 7 compares the linear jerk of J-NNC with that of the configuration without KCM on the same representative path.

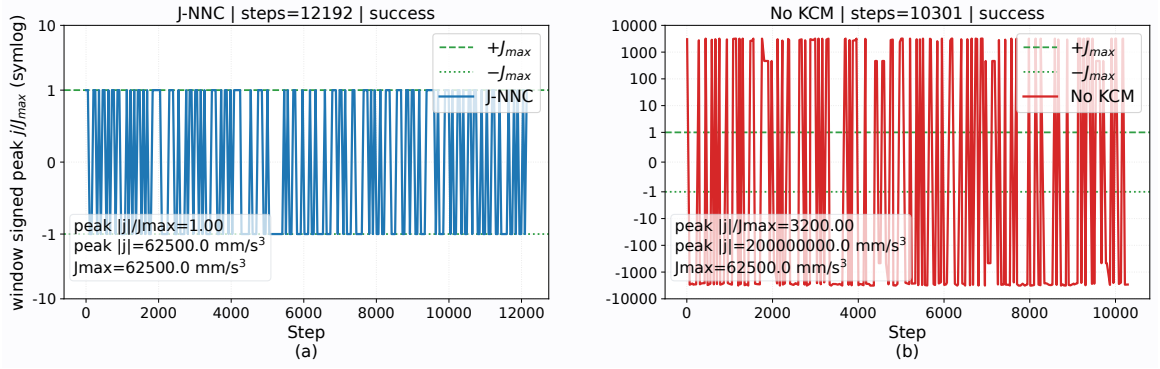

Fig. 7: Linear-jerk constraint comparison on the same representative path: (a) J-NNC with KCM; (b) ablated configuration without KCM.

As shown in Fig. 7(a), the linear jerk of J-NNC stays within the bound defined by $J_{\max}$. In Fig. 7(b), removing KCM produces high-amplitude jerk spikes. This result confirms that faster traversal or smaller contour error alone is insufficient for CNC interpolation; the learned command must also be projected onto the feasible kinematic set.

The remaining ablations give a more nuanced view of Adaptive Preview and reward design. The fixed Adaptive Preview setting is feasible and, in the reported runs, can be faster than the state-dependent setting. This result does not weaken the role of KCM, but it indicates that the present Adaptive Preview parameterization has not yet fully converted state-dependent Adaptive Preview selection into a consistent speed advantage. Its value is that the policy can vary the forward geometric range according to the local state; the benefit of this design should become clearer with improved parameterization and more diverse paths. Removing Adaptive Preview observation still leaves enough local tracking and corner information to finish the paths, but it reduces access to upcoming geometric trends that can support earlier steering and speed adjustment. Removing the straight-line/corner dual-reward scheme changes the balance among progress, contour regulation, and smoothness, because straight feeding and corner transition require different control priorities.

Overall, the ablation results separate the roles of the main components. KCM is the component that guarantees dynamic feasibility. Adaptive Preview and phase-dependent rewards affect how the feasible commands are selected within the tolerance band, although the current Adaptive Preview design still needs refinement to provide a more consistent advantage over fixed settings. Future improvements should therefore focus on Adaptive Preview parameterization and velocity continuity after corner traversal rather than on relaxing the execution-side jerk projection.

5. Conclusion

This paper presented J-NNC, a reinforcement learning-based CNC motion-planning method for mixed CAM-generated tool paths subject to contour-tolerance and kinematic constraints. The main difficulty considered in this work is that geometric smoothing, feedrate planning, and dynamic feasibility cannot be treated independently in practical CNC interpolation. A trajectory that stays close to the reference polyline may still contain sharp direction changes near corners, while a learned controller that advances the path efficiently may produce commands that exceed velocity, acceleration, or jerk limits. J-NNC addresses this issue by treating CNC interpolation as a closed-loop constrained decision-making process, rather than as a serial procedure with separate smoothing and feedrate-planning stages. The structured state and Adaptive Preview action allow the policy to combine tracking, kinematic, corner, and forward-geometric information when planning local transitions within the tolerance band. The KCM further projects learned steering and feedrate outputs into executable commands satisfying velocity, acceleration, and jerk limits. Experiments on square, circular, and butterfly-shaped paths show full path progress and zero maximum relative linear-jerk violation, confirming the importance of explicit action projection compared with the NNC baseline and the no-KCM ablation.

Future work will refine the Adaptive Preview action and corner-phase representation to improve traversal efficiency and velocity continuity after corner transitions. Further validation will include servo lag, axis-level constraints, and real machining experiments to assess practical deployment on CNC systems.

References

[1] Yuwen Sun, Jinjie Jia, Jinting Xu, Mansen Chen, and Jinbo Niu. Path, feedrate and trajectory planning for free-form surface machining: A state-of-the-art review. *Chinese Journal of Aeronautics*, 35(8):12–29, 2022.

[2] Guangwen Yan, Desheng Zhang, Jinting Xu, and Yuwen Sun. Corner smoothing for CNC machining of linear tool path: A review. *Journal of Advanced Manufacturing Science and Technology*, 3(2):2023001, 2023.

[3] Kaan Erkorkmaz and Yusuf Altintas. High speed CNC system design. part I: jerk limited trajectory generation and quintic spline interpolation. *International Journal of Machine Tools and Manufacture*, 41(9):1323–1345, 2001.

[4] A. C. Lee, M. T. Lin, Y. R. Pan, and W. Y. Lin. The feedrate scheduling of NURBS interpolator for CNC machine tools. *Computer-Aided Design*, 43(6):612–628, 2011.

[5] Burak Sencer, Kazunori Ishizaki, and Eiji Shamoto. High speed cornering strategy with confined contour error and vibration suppression for CNC machine tools. *CIRP Annals*, 64(1):369–372, 2015.

[6] S. Tajima and B. Sencer. Kinematic corner smoothing for high speed machine tools. *International Journal of Machine Tools and Manufacture*, 108:27–43, 2016.

[7] Yong Zhang, Hui Zhang, Jiadong Shi, Jiali Jiang, and Mingyong Zhao. Development of an adaptive-feedrate planning and iterative interpolator for parametric toolpath with normal jerk constraint. *International Journal of Precision Engineering and Manufacturing*, 22:73–81, 2021.

[8] Yifei Hu, Xin Jiang, Guanying Huo, Cheng Su, Hexiong Li, Li-Yong Shen, and Zhiming Zheng. Enhancing five-axis CNC toolpath smoothing: Overlap elimination with asymmetrical B-splines. *CIRP Journal of Manufacturing Science and Technology*, 52:36–57, 2024.

[9] W. Fan, C. Lee, and J. Chen. A realtime curvature-smooth interpolation scheme and motion planning for CNC machining of short line segments. *International Journal of Machine Tools and Manufacture*, 96:27–46, 2015.

[10] Y. Zhang, P. Ye, J. Wu, and H. Zhang. An optimal curvature-smooth transition algorithm with axis jerk limitations along linear segments. *The International Journal of Advanced Manufacturing Technology*, 95:875–888, 2018.

[11] Bingran Li, Hui Zhang, Peiqing Ye, and Jinsong Wang. Trajectory smoothing method using reinforcement learning for computer numerical control machine tools. *Robotics and Computer-Integrated Manufacturing*, 61:101847, 2020.

[12] De-Ning Song, Jian-Wei Ma, Yu-Guang Zhong, and Jian-Jun Yao. Global smoothing of short line segment toolpaths by control-point-assigning-based geometric smoothing and FIR filtering-based motion smoothing. *Mechanical Systems and Signal Processing*, 160:107908, 2021.

[13] Z. Yang, L. Shen, C. Yuan, and X. Gao. Curve fitting and optimal interpolation for CNC machining under confined error using quadratic B-splines. *Computer-Aided Design*, 66:62–72, 2015.

[14] Bingcai Wu, Junyan Ma, Lutao Wei, Xiaoping Liao, and Juan Lu. NURBS interpolator with scheduling scheme combining cubic and quartic S-shaped feedrate profiles under drive and chord error constraints. *Computer-Aided Design*, 152:103380, 2022.

[15] M. Tsai, H. Nien, and H. Yau. Development of a real-time look-ahead interpolation methodology with spline-fitting technique for high-speed machining. *The International Journal of Advanced Manufacturing Technology*, 47:621–638, 2010.

[16] De-Ning Song, Yu-Guang Zhong, and Jian-Wei Ma. Look-ahead-window-based interval adaptive feedrate scheduling for long five-axis spline toolpaths under axial drive constraints. *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture*, 234(13):1656–1670, 2020.

[17] Huan Zhao, Limin Zhu, and Han Ding. A real-time look-ahead interpolation methodology with curvature-continuous B-spline transition scheme for CNC machining of short line segments. *International Journal of Machine Tools and Manufacture*, 65:88–98, 2013.

[18] M. Tsai and Y. Huang. A novel integrated dynamic acceleration/deceleration interpolation algorithm for a CNC controller. *The International Journal of Advanced Manufacturing Technology*, 87:279–292, 2016.

[19] Molong Duan and Chinedum Okwudire. Minimum-time cornering for CNC machines using an optimal control method with NURBS parameterization. *The International Journal of Advanced Manufacturing Technology*, 85:1405–1418, 2016.

[20] B. Li, H. Zhang, and P. Ye. Driving force constraint corner smoothing method for high-speed machine tools. In *ASME 2017 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, page V006T10A012, 2017.

[21] Hexiong Li, Xin Jiang, Guanying Huo, Cheng Su, Shiwei Zhou, Bolun Wang, Yifei Hu, and Zhiming Zheng. An integrated trajectory smoothing method for lines and arcs mixed toolpath based on motion overlapping strategy. *Journal of Manufacturing Processes*, 95:242–265, 2023.

[22] S. Sun, P. Zhao, T. Zhang, B. Li, and D. Yu. Smoothing interpolation of five-axis tool path with less feedrate fluctuation and higher computation efficiency. *Journal of Manufacturing Processes*, 109:669–693, 2024.

[23] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

[24] Burak Sencer, Kosuke Ishizaki, and Eiji Shamoto. A curvature optimal sharp corner smoothing algorithm for high-speed feed motion generation of NC systems along linear tool paths. *The International Journal of Advanced Manufacturing Technology*, 76(9–12):1977–1992, 2015.